软件构造期末复习 byLPA

一、单项选择题（每道 2 分）

1.下列哪一项最能体现“开闭原则”(Open/Closed Princaiple)的精髓? （C）

A.将所有类设计为 final，防止继承

B.通过继承扩展功能，而非修改现有代码

C.通过组合和接口抽象，在不修改现有类的前提下扩展行为

D.所有方法都必须是 public，以便外部调用

2.在使用策略模式 (Strategy Pattern）时，若客户端需要在运行时动态切换算法，但算法之间共享大量公共状态，最应避免的做法是 （B）

A.将共享状态封装在上下文(Context)中

B.将共享状态作为策略类的成员变量，每个策略实例独立维护

C.使用工厂模式创建策略实例，避免重复初始化

D.将共享状态提取为静态变量，供所有策略共享。

3.以下哪项重构技术最适用于“Long Method”（过长方法）的代码坏味道? （C）

A. Extract Class

B. Replace Conditional with Polymorphism

C. Extract Method

D. Introduce Parameter Object

4.当一个类同时承担了“数据存储”和“业务逻辑计算”职责，违反了以下哪个设计原则? （A）

A.单一职责原则（SRP）

B.里氏替换原则（LSP）

C.接口隔离原则（ISP）

D.依赖倒置原则（DIP）

5.在观察者模式中，若被观察者 (Subject) 在通知观察者时抛出异常，导致部分观察者未收到通知，以下哪种改进最符合“健壮性”与“解耦”原则? （C）

A.在 Subject 中使用 try-catch 包裹所有通知调用

B.将观察者列表复制后遍历，避免并发修改异常

C 引入“消息队列”异步通知，观察者失败不影响主体

D.要求所有观察者必须实现 ErrorHandling 接口

二、简答题（每道 5 分)

6.说明以下设计的问题；重构使其满足 OCP，画类图说明你的重构思路；对照类图，实现代码

```
public class DocumentProcessor {
    public Document createDocument(String type) {
        if ("PDF". equals(type)){
            return new PdfDocument();
        } else if ("WORD".equals(type)){
            return new WordDocument();
        } else if ("EXCEL".equals(type)){
            return new ExcelDocument();
        }
        return null;
    }
}
```

提示：工厂方法

7.说明以下设计的问题；重构使其满足 OCP，画类图说明你的重构思路；对照类图，实现代码

```
public class Calculator {
    public void compute(String operation, double a, double b) {
        if (operation.equals("add")){
            System.out.println(a + b);
        }
        else if (operation.equals("subtract")) {
            System.out.println(a - b);
        }else if (operation.equals("multiply")) {
            System.out.println(a * b);
        }else if (operation.equals("divide")){
            if (b ==0){
                System.out.println("错误：除数不能为零");
            } else {
                System.out.println(a / b);
            }
        }else {
            System.out.println("不支持的操作");}
        }
    }
}
```

提示：策略

8.说明以下设计的问题；重构使其满足 OCP，画类图说明你的重构思路；对照类图，实现代码

```
public class OrderService {
    public void createOrder(Order order){
        // 1.保存订单
        orderRepository.save(order);
        // 2.发送邮件
        emailService.sendConfirmation(order.getEmail());
```

```
        // 3. 发送短信
        smsService.send(order.getPhone(), "订单已创建");
        // 4. 更新库存
        inventoryService.decrease(order.getItems());
        // 5. 记录日志
        logger.info("Order " + order.getId() + " created");
    }
}
```

提示：责任链/状态模式

9.说明以下设计的问题；重构使其满足 OCP，画类图说明你的重构思路；对照类图，实现代码

```
public class ReportGenerator {
    public void generateMonthlyReport() {
        System.out.println("=== 月度报告 ===");
        loadData();
        validateData();
        calculateMetrics();
        formatOutput();
        saveToFile();
    }
    public void generateWeeklyReport() {
        System.out.println("=== 周度报告 ===");
        loadData();
        validateData();
        calculateMetrics();
        formatOutput();
        saveToFile();
    }
}
```

提示：模板方法

10.说明以下设计的问题；重构使其满足 OCP，画类图说明你的重构思路；对照类图，实现代码

```
public class RiskApprovalService {
    public boolean approve(Transaction tx) {
        if (tx.getAmount() < 1000) return true;
        if (tx.getAmount() < 5000 && !isFraudulent(tx)) return true;
        if (tx.getAmount() < 20000 && isFraudulent(tx)) {
            return approveByManager(tx);
        }
        if (tx.getAmount() >= 20000) {
            return approveByManager(tx) && approveByCompliance(tx) && approveByCFO(tx);
        }
        return false;
    }
}
```

基于交易金额的多级审批流程：金额小于 1000 直接通过；金额 1000-4999 需通过非欺诈检测；金额 5000-19999 需通过经理审批；金额 20000 及以上需通过经理、合规和首席财务官三重审批

提示：责任链

11.说明以下设计的问题；重构使其满足 OCP，画类图说明你的重构思路；对照类图，实现代码

```
public class ShoppingService{
    public void shipWithSF(String tracking, double weight) {...}// 顺丰
    public void shipWithJD(String id, double weight) {...} // 京东
    public void shipWithYTO(String code, double weight) {...} // 韵达
}
```

提示：适配器 + 策略

12.请解释“组合优于继承”(Composition over Inheritance）原则，并结合一个具体例子（如“动物”与“飞行能力”）说明：为何使用组合（如 FlyBehavior 接口）比多层继承（如 FlyingDuck,FlyingBird）更符合面向对象设计原则？

提示：Lec03

13.“状态模式”与“策略模式”在结构上相似（都涉及抽象父类与具体子类），但设计意图和适用场景截然不同。请从“控制权归属”和“变化点”两个维度对比分析二者的核心差异，并各举一个现实系统中的应用场景。

三、设计与分析题（20 分）

14.为店铺商家设计一个穿搭系统，可以让顾客模拟穿搭效果。支持动态添加 T 恤、裤子、鞋子等服饰，并展示最终搭配效果。要求：

-（1）绘制类图，并说明设计思路

-（2）分析你的设计如何满足 OCP。

-（3）根据类图，编写代码框架

-（4）编写测试类，完成最终搭配效果展示

提示：装饰模式

15.你正在为一个电商系统设计订单处理模块。系统需支持：

■多种支付方式（信用卡、支付宝、微信）

■多种配送方式（标准快递、次日达、自提）

■多种促销策略（满减、折扣券、积分抵扣）

■订单状态机（待支付→已支付→已发货→已完成→已取消）

要求：

a) 设计一个高内聚、低耦合的系统架构；

b) 画出核心类图（文字描述即可，需包含类名、关键方法、关系）；

c) 说明如何保证系统在新增支付方式或促销策略时无需修改已有代码；

d) 指出你设计中可能存在的潜在扩展瓶颈，并提出改进思路。

提示：策略

16.以下是一个“订单处理”类的初始实现（Java 伪代码），存在严重的设计缺陷和代码坏味道。请对其进行系统性重构，要求：

①明确指出至少 5 个代码坏味道(如:Long Method、Switch Statement、Feature Envy、Data Clumps 等）；

②使用设计模式和重构技术进行改进；

③给出重构后的核心类结构（类名、职责、关系）；

④说明重构后如何满足 SOLID 原则；

画出重构前后的类图对比（需要对类图进行文字描述）

```
public class OrderProcessor {
    private String customerName;
    private List<Item> items;
    private String paymentMethod; // "CREDIT","PAYPAL", "CRYPTO"
    private String shippingType;  //"STANDARD","EXPRESS""SELF_PICKUp"
    private double total;
    public void processOrder() {
        // 1. 校验订单
        if (items.isEmpty()) throw new IllegalArgumentException("No items");
        if(customerName == null | customerName.trim().isEmpty())
            throw new IllegalArgumentException("Invalid customer");
        // 2. 计算总价
        total =0;
        for (Item item : items) {
            total += item.getPrice() *item.getQuantity();
        }
        // 3. 应用促销（根据商品类型）
        if (items.stream().anyMatch(i ->i.getType().equals("BooK")) {
            total *= 0.9;//书打 9 折
        }
            if (total > 1000) {
                total-=100; //满 1000 减 100
            }
        // 4.支付处理(switch)
        if (paymentMethod.equals("CREDIT")){
            CreditcardProcessor ccp = new CreditcardProcessor();
```

```
            if (!ccp.charge(total)) throw new PaymentException("Credit card failed");
        } else if (paymentMethod.equals("PAYPAL")) {
            PayPalProcessor ppp = new PayPalProcessor();
            if(!ppp.pay(total)) throw new PaymentException("PayPal failed");
        } else if (paymentMethod.equals("CRYPTO")) {
            CryptoProcessor cpp = new CryptoProcessor();
            if(!cp.transfer(total)) throw new PaymentException("Crypto failed");
        } else {
            throw new IllegalArgumentException("Unknown payment method");
        }
        // 5. 配送处理(switch)
        if (ShippingType.equals("STANDARD")) {
            StandardShipping ss = new StandardShipping();
            ss.scheduleDelivery();
        } else if (shippingType.equals("EXPRESS")) {
            ExpressShipping es = new ExpressShipping();
            es.scheduleDelivery();
        } else if (shippingType.equals("SELF_PICKUP")){
            SelfPickup sp = new SelfPickup();
            sp.notifyReady();
        } else {
            throw new IllegalArgumentException("Unknown shipping type");
        }
        // 6. 发送邮件
        EmailService email = new GmailService();
        email.send("order confirmed for"+ customerName);
        // 7. 记录日志
        Logger.log("Order processed:"+ total + " by "+ customerName);
    }
}
```